Student Performance Analytics System

Name: YASH KUMAR
Course: Python with AI

1. Project title

Student Performance Analytics System

2. Objective

The project calculates student results from a CSV file. Checking totals, grades and subject performance by hand becomes repetitive as the class grows. This program reads the records once and prepares a report showing individual results and the overall performance of the class.

3. Technologies used

- Python for functions, loops, conditions, file handling and the command-line program.
- NumPy for totals, averages, minimums, maximums, medians and pass checks.
- Pandas for CSV loading, DataFrames, filtering, sorting and CSV export.
- Python's built-in unittest module for automated checks.

The project has two external dependencies: NumPy 2.3.5 and Pandas 3.0.1. It does not train a machine learning model; the task is descriptive data analysis.

4. Dataset description

The dataset contains 24 fictional student records and nine columns. Each record has a unique Student_ID, Name, Department, five subject marks and Attendance. The departments are BCA, BSc CS and BSc IT. The subjects are Python, Mathematics, Statistics, English and Computer_Basics. Each mark is out of 100 and attendance is a percentage from 0 to 100.

The sample includes strong and weak results, a mark of exactly 40, a high-average student with one failed subject, low attendance and tied averages. These records make it possible to check the main decisions in the program. They are examples created for this project and do not describe real students.

5. Implementation

Step 1: Load and validate

`load_students()` checks the CSV headers and row lengths before reading the file with `pandas.read_csv()`. It keeps identifiers as text so leading zeros are not lost. Spaces around values are removed. Missing fields, duplicate student IDs, non-numeric marks and values outside 0 to 100 are rejected with a clear error. Extra columns are ignored. Invalid records are not silently removed.

Step 2: Calculate individual results

`calculate_results()` copies the DataFrame and converts the five subject columns to a NumPy array. `np.sum(..., axis=1)` adds the marks in each row and `np.mean(..., axis=1)` finds each student's average.

For example, Meera Nair has marks of 96, 94, 95, 91 and 94:

```
Total   = 96 + 94 + 95 + 91 + 94 = 470 out of 500
Average = 470 / 5 = 94.00 out of 100
Result  = Pass, because every subject is at least 40
Grade   = A+
```

`np.all(marks >= 40, axis=1)` checks every subject in a row. A student must pass all five subjects to get an overall Pass. `assign_grade()` uses the rules below:

Condition	Grade
All subjects passed; average at least 90	A+
All subjects passed; average 80 to below 90	A
All subjects passed; average 70 to below 80	B
All subjects passed; average 60 to below 70	C
All subjects passed; average 50 to below 60	D
All subjects passed; average 40 to below 50	E
Any subject below 40	F

All subjects carry equal weight. The grade thresholds and pass mark are choices for this implementation because the assignment asks for a clearly defined rule. Attendance below 75% is flagged as Low, independently of pass/fail. Calculations use full precision; rounding to two decimals happens only when presenting or exporting results.

Step 3: Analyse the class and subjects

`class_summary()` finds the number of students, pass/fail counts, pass percentage, class mean, highest and lowest student averages, median and attendance figures. The pass percentage is the number passing all subjects divided by the number of students, multiplied by 100.

`subject_analysis()` loops over the five subject names. For each subject it finds the highest, lowest and average mark, counts passes and failures, and calculates the subject pass percentage. Subject pass rates consider only that subject; the overall pass rate requires all subjects to be passed.

Step 4: Find top performers

`top_performers()` ranks all students by their unrounded average. Tied averages share a rank, and the next rank skips the occupied places. With the sample data, the ranks are 1, 2, 2, 4, 5 and 5. Both students tied for fifth are included in the top-five report. Pass/fail is shown alongside the ranking.

Step 5: Display and save

`build_report()` formats the loaded data and results for the terminal. `save_reports()` writes the same text report and five CSV tables to `reports/`. The program can also accept another input file, another output folder and a different number of top students through command-line arguments.

6. Key features

- Individual totals, averages, grades, pass/fail results and failed-subject lists.
- Class statistics and subject-wise performance.
- Top performers with ties handled consistently.
- Grade distribution and a separate low-attendance list.
- CSV validation and understandable error messages.
- Text and CSV reports that can be checked without reading the source code.

7. Output

Running `python main.py` with the supplied dataset gives:

Measure	Value
Number of students	24
Passed	18
Failed	6
Overall pass percentage	75.00%
Class average	69.34
Highest average	Meera Nair, 94.00
Lowest average	Om Prakash, 33.00
Median average	73.50
Average attendance	82.67%
Low-attendance students	6

Subject	Highest	Lowest	Average	Passed	Failed
Python	100	28	67.79	20	4
Mathematics	94	25	65.04	20	4
Statistics	95	33	67.67	22	2
English	94	42	73.62	24	0
Computer_Basics	96	35	72.58	22	2

English has the highest subject average. Mathematics has the lowest subject average, so its results deserve closer attention. The full output is included in `reports/analysis_report.txt`; the other report files contain the same analysis in CSV form.

8. Final outcome

The program reads the dataset, validates it and produces the required individual, class and subject results. The supplied class has a 75% pass rate. Six students have at least one failed subject, and six have attendance below 75%; these are separate groups, with some overlap.

All 22 automated tests passed on Python 3.12.14 and Python 3.14.6 on Windows. The checks cover expected calculations, grade and attendance boundaries, tied scores, invalid input, report exports and running from another folder. The Windows launcher also ran successfully, and the installed packages passed `pip check` without dependency conflicts.

This is an offline command-line program for one dataset at a time. It assumes five equally weighted subjects out of 100. It does not store records in a database, predict future performance or apply attendance-based exam eligibility. Report files in the selected output folder are replaced on the next run.

9. Implementation challenges and concepts to understand

A high average can hide a failed subject. Dev Malhotra has an average of 80.60 but a Mathematics mark of 39. Checking every subject before assigning a grade prevents an incorrect Pass result.

Ties should not be lost. Taking only the first five sorted rows would exclude one student who shares fifth place. Ranking first and keeping all rows up to rank five includes both students.

CSV data needs checking. Blank marks, repeated IDs and extra separators can change the analysis. The loader rejects those inputs before calculating results.

Rounding belongs at the output stage. An average just below a grade boundary must keep its original grade even if its displayed value rounds up.

The code demonstrates row-wise NumPy operations, Pandas filtering and sorting, functions that separate calculation from display, and tests for boundary cases. These are the main concepts to be ready to explain during assessment.

10. GitHub repository and submission

Repository URL: not yet published.

Upload the source files, dataset, README, this documentation, tests and sample reports to a repository named `student-performance-analytics` or another clear name. Exclude the local `.venv` folder and Python cache files. Replace the status above with the actual repository URL after publishing and submit that URL through the EWB Google Form supplied by the trainer. Neither a GitHub repository nor a Google Form submission is part of the current local deliverable.

The source assignment lists **15 October 2026** as the deadline. Any later submission arrangement must be confirmed with the trainer.